

Pod Security Validation

Detailed validation of Kubernetes Pod Security restricted profile, hardened security contexts, read-only filesystem behavior, privileged pod blocking, and CIS Benchmark alignment.

Field	Value
Document Type	Pod Security Validation
Author	Jagriti Banerjee
Project Scope	Private Kubernetes DevSecOps lab using Docker, Kind, Calico, Trivy, Falco, OPA/Gatekeeper, ArgoCD, and kube-bench
Audience	Security engineers, DevSecOps reviewers, recruiters, and technical interviewers
Classification	Private lab / portfolio evidence

Reviewer snapshot

Review Dimension	Included in Document
Technical depth	Control rationale, validation commands, expected results, risk interpretation, and remediation guidance.
Evidence model	Screenshot references, manifest fields, scanner outputs, and repeatable kubectl commands.
Professional use	Can be used as a GitHub portfolio artifact, interview talking point, and internal-style security record.
CIS alignment	Includes a dedicated CIS Kubernetes Benchmark section with lab-context interpretation.

Scope statement

This document is based on controlled, authorized testing performed inside a private lab environment. It does not represent a production certification and does not include public or third-party targets. The focus is professional evidence, repeatable validation, and security engineering reasoning.

Spacing and readability note

The document is intentionally laid out with compact paragraphs, split tables, and dedicated evidence pages so that detail is preserved without crowded pages or large blank gaps.

Document coverage

Coverage Area	Included
Assessment narrative	Executive summary, scope, methodology, technical analysis, and validation interpretation.
Evidence references	Relevant screenshots, command outputs, reports, and expected retest outcomes.
Remediation planning	Priority-based actions, production notes, and follow-up recommendations.
Benchmark context	CIS Kubernetes Benchmark alignment and lab-specific interpretation.

1. Executive Summary

This document validates workload-level security controls for the Enterprise DevSecOps demo application. It confirms that the application is constrained through namespace Pod Security labels, pod securityContext, container securityContext, read-only filesystem behavior, and privileged pod blocking. The testing demonstrates both secure state and attempted unsafe state rejection.

2. Controls Validated

Control	Expected State	Why It Matters
Restricted namespace labels	enforce/warn/audit set to restricted	Blocks broad categories of unsafe pod settings.
Non-root execution	runAsNonRoot true and UID 10001	Reduces impact of application compromise.
No privilege escalation	allowPrivilegeEscalation false	Prevents gaining extra Linux privileges.
Dropped capabilities	capabilities.drop ALL	Removes unnecessary kernel-level privileges.
RuntimeDefault seccomp	seccompProfile RuntimeDefault	Applies default syscall filtering.
Read-only root filesystem	readOnlyRootFilesystem true	Reduces tampering and persistence.
Privileged pod rejection	Privileged hostPath pod is forbidden	Blocks a common container escape pattern.

3. Validation Commands

```
kubectl get ns devsecops --show-labels
kubectl get pod -n devsecops -l app=demo-app -o jsonpath='{.items[0].spec.securityContext}'
kubectl get pod -n devsecops -l app=demo-app -o jsonpath='{.items[0].spec.containers[0].securityContext}'

POD=$(kubectl -n devsecops get pod -l app=demo-app -o jsonpath='{.items[0].metadata.name}')
kubectl -n devsecops exec "$POD" -- sh -c 'touch /app/test.txt'
kubectl -n devsecops exec "$POD" -- sh -c 'touch /tmp/test.txt && echo tmp-write-ok'

kubectl apply -f redteam/runtime/privileged-hostpath-pod.yaml
```

4. Privileged Pod Attack Pattern

The lab intentionally attempted to deploy a privileged pod with hostPath access to demonstrate why Pod Security enforcement matters. A privileged pod combined with a host root mount can expose node filesystem access and create a container escape pattern. After remediation, the restricted namespace policy blocks this pod before it runs.

Risk	Remediation
privileged: true removes major container isolation boundaries.	Enforce restricted Pod Security and block privileged containers.
hostPath path / exposes node filesystem to the container.	Avoid hostPath; use safer volume types and admission policies.
Root execution increases blast radius.	Run as non-root fixed UID/GID.
Writable root filesystem enables tampering.	Use read-only root filesystem with explicit writable mounts.

5. CIS Kubernetes Benchmark Section

CIS describes the Kubernetes Benchmark as consensus-based secure configuration guidance for Kubernetes. In this lab, CIS Benchmark alignment is used as an assessment reference and learning framework. Because the cluster is a Kind-based private lab rather than a production kubeadm or managed service environment, benchmark results are interpreted with environment context. The expected professional practice is to document PASS, FAIL, WARN, and not-applicable outcomes; assign owners to applicable failures; retest after remediation; and keep benchmark artifacts in the evidence folder.

Pod Security validation maps to benchmark-aligned workload hardening themes such as preventing privileged containers, reducing unnecessary privileges, and limiting access to host resources. kube-bench provides configuration review support, while this document adds live workload behavior validation.

6. Evidence Highlights

7. Retest Checklist

Retest	Pass Criteria
Namespace labels	restricted labels are present and not accidentally removed.
Non-root settings	jsonpath shows runAsNonRoot true and UID 10001.
Privilege escalation	container securityContext shows allowPrivilegeEscalation false.
Capabilities	container drops ALL capabilities.
Filesystem	touch /app/test.txt fails; touch /tmp/test.txt succeeds.
Privileged pod	attempted privileged hostPath pod is rejected.

Quality Assurance and Evidence Preservation

QA Item	Required Practice
Evidence naming	Use stable screenshot names and link each item to a control or retest step.
Command repeatability	Store commands in documentation so another reviewer can reproduce validation.
Exception handling	Any accepted risk should include owner, reason, expiry date, and compensating control.
Retest discipline	Do not close a remediation item until the expected negative and positive tests both pass.
CI/CD linkage	Where possible, convert manual validation into security gates or scheduled checks.

Reviewer notes for this Kubernetes security area

This document intentionally combines implementation evidence with risk interpretation. The objective is to show not only that commands were executed, but also why each control matters, how it reduces attack surface, and how the result should be retested after future changes.

References and Standards Basis

Reference	Use in this Document
CIS Kubernetes Benchmark	CIS describes its Kubernetes Benchmark as a product of a community consensus process with secure configuration guidelines for Kubernetes.
Kubernetes RBAC documentation	Kubernetes RBAC uses Role, ClusterRole, RoleBinding, and ClusterRoleBinding objects to manage authorization.
Kubernetes NetworkPolicy documentation	NetworkPolicy governs how pods communicate with each other and with network endpoints.
Kubernetes Pod Security Standards	The restricted profile is intended for strongly hardened workload execution.

Reference URLs used during document preparation: <https://www.cisecurity.org/benchmark/kubernetes>, <https://kubernetes.io/docs/reference/access-authn-authz/rbac/>, <https://kubernetes.io/docs/concepts/services-networking/network-policies/>, and <https://kubernetes.io/docs/concepts/security/pod-security-standards/>.